



BITBOYS

Open GLES 2.0

프로그램 가능 파이프라인 소개

Robert J. Simpson
Bitboys Oy. 수석 설계자

개요

- **ES 2.0 철학**
- 셰이더 모델
- **GLSL(OpenGL Shading Language)**
- 셰이더 예제
- 셰이더 컴파일
- 성능 팁
- 셰이더의 미래

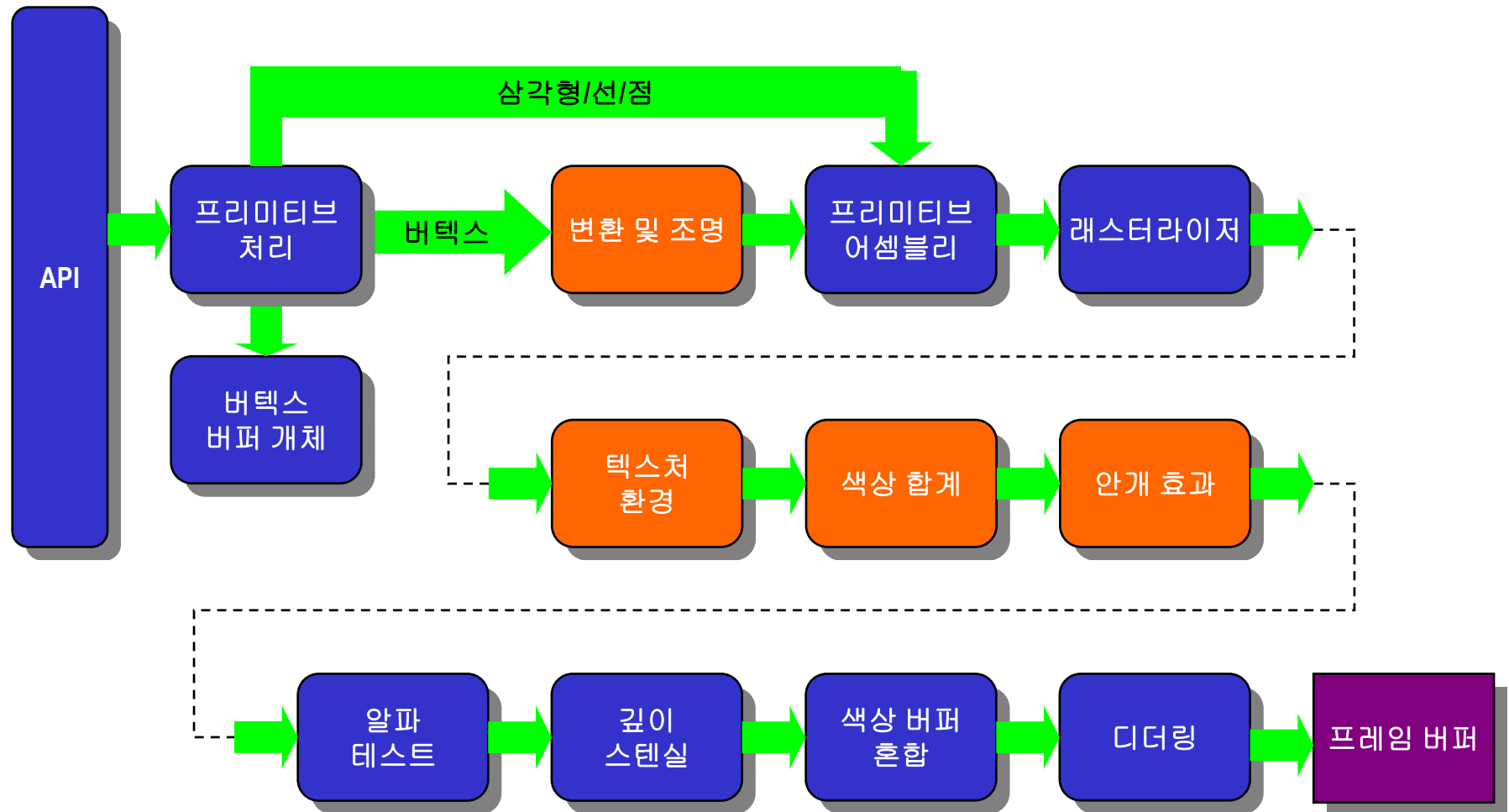
Bitboys: 기술 혁신의 역사

- 14년의 하드웨어 그래픽 솔루션 개발 경험
- Bitboys 팀의 반도체 개발 경험:
 - 12개의 칩 제작
- 그래픽 프로세서 설계와 관련된 다양한 기술 혁신
 - 1995년, 최초로 버텍스 셰이더와 픽셀 셰이더를 단일 칩에 통합 – 현재 그래픽 프로세서의 가장 중요한 기술 중 하나
 - 1995년, 최초로 다중 텍스처 하드웨어를 지원하는 단일 칩 개발
 - 1998년, Microsoft DirectX 6.0 드라이버에 그래픽 기술 사용권 부여
 - 2003년, NEC와 협력하여 최초의 벡터 그래픽 하드웨어 가속기 개발
- **2002년부터 무선 장치와 임베디드 장치용 그래픽 솔루션 개발에 주력**

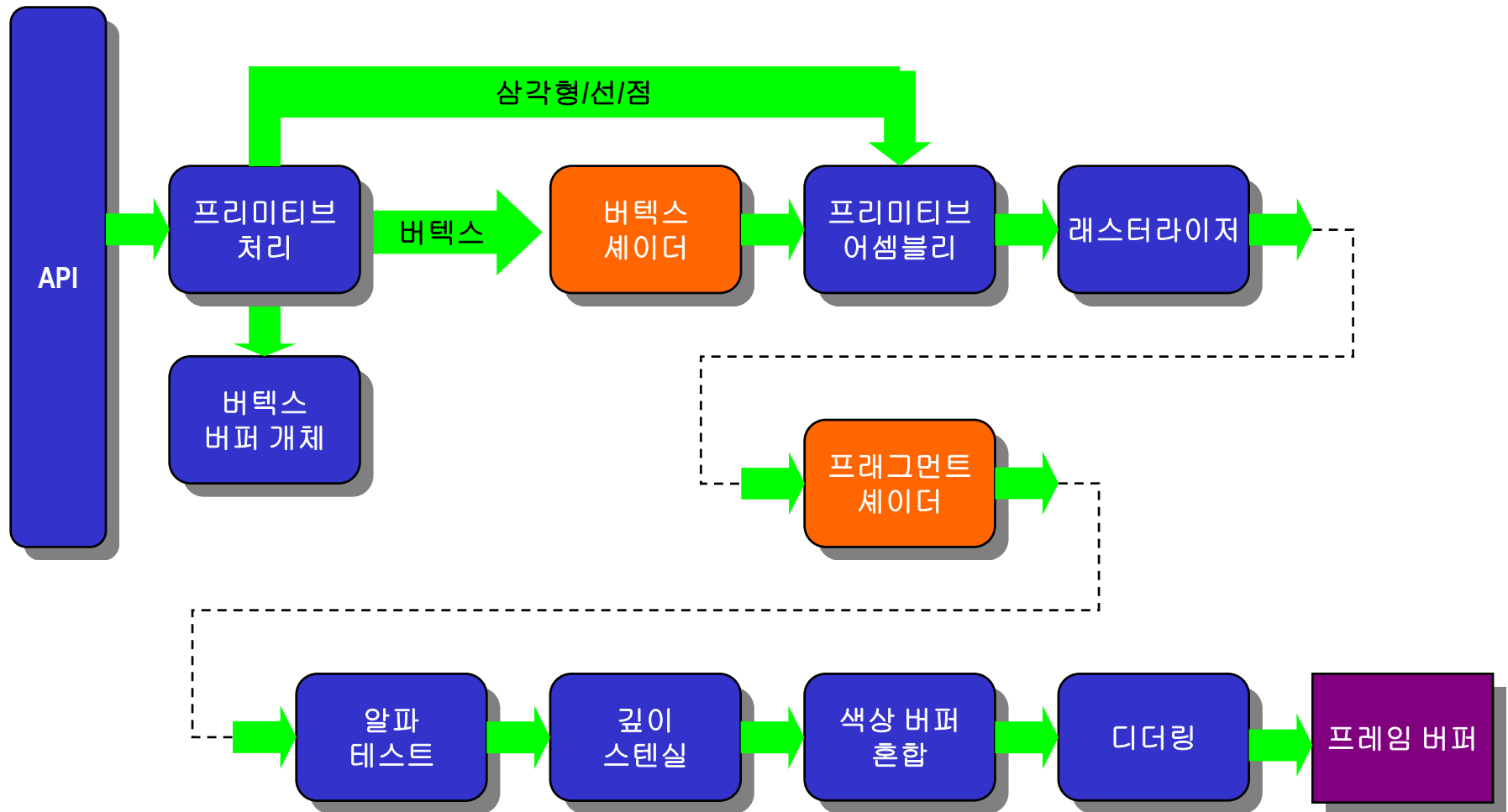
ES 2.0 철학

- **OpenGL 2.0에 사용되는 GLSL 기반**
 - 개방형 표준
 - 데스크톱에서 성능 입증
- **임베디드 장치에 사용하도록 최적화**
 - 셰이더 프로그램 크기 감소
 - 레지스터 사용량 최소화
 - 수치 정밀도 축소
- **완전한 프로그램 가능 모델**
 - 대부분의 고정된 기능 제거
- **ES1.x와의 100% 후방 호환성은 보장 못함**

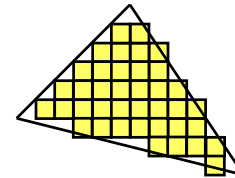
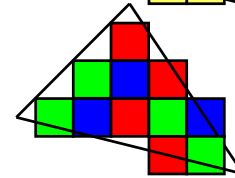
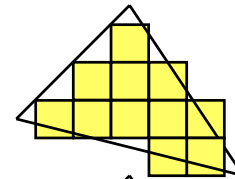
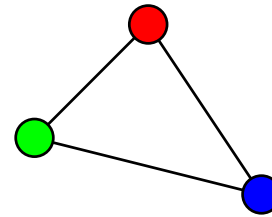
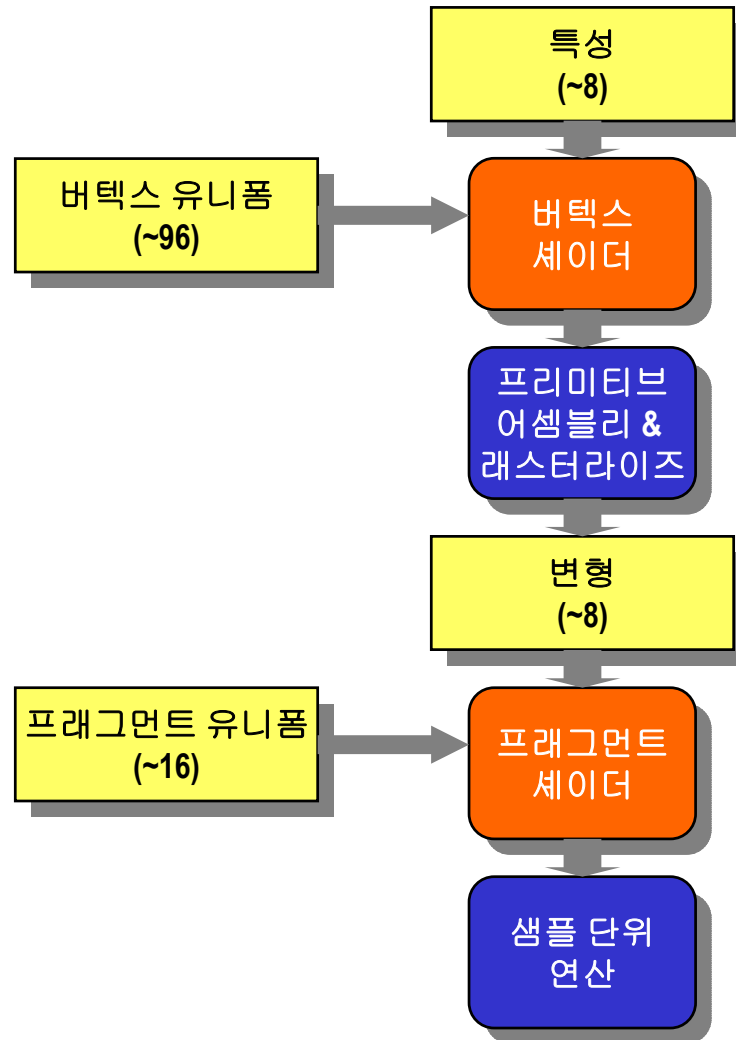
기존의 고정 함수 파이프라인



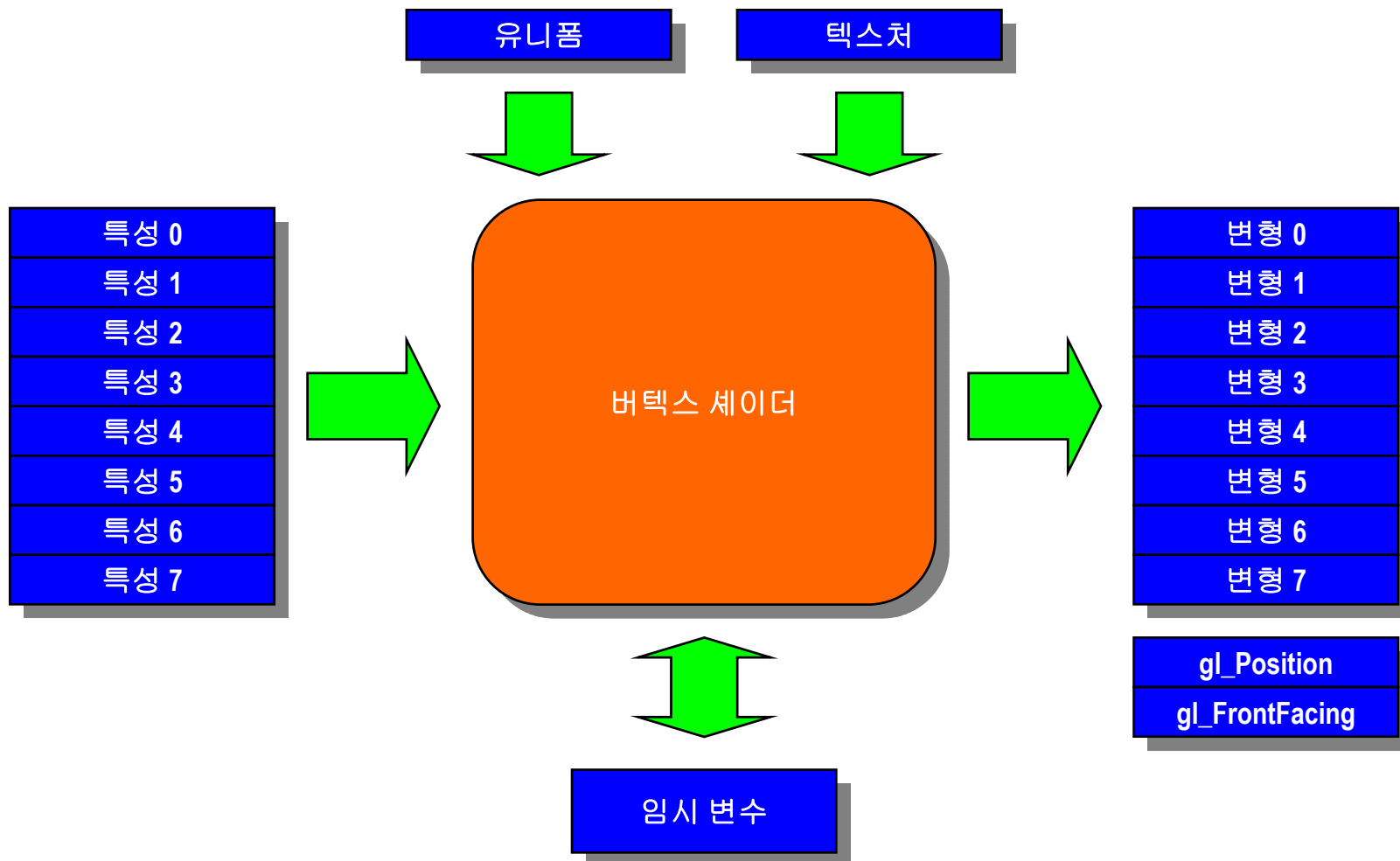
ES2.0 프로그램 가능 파이프라인



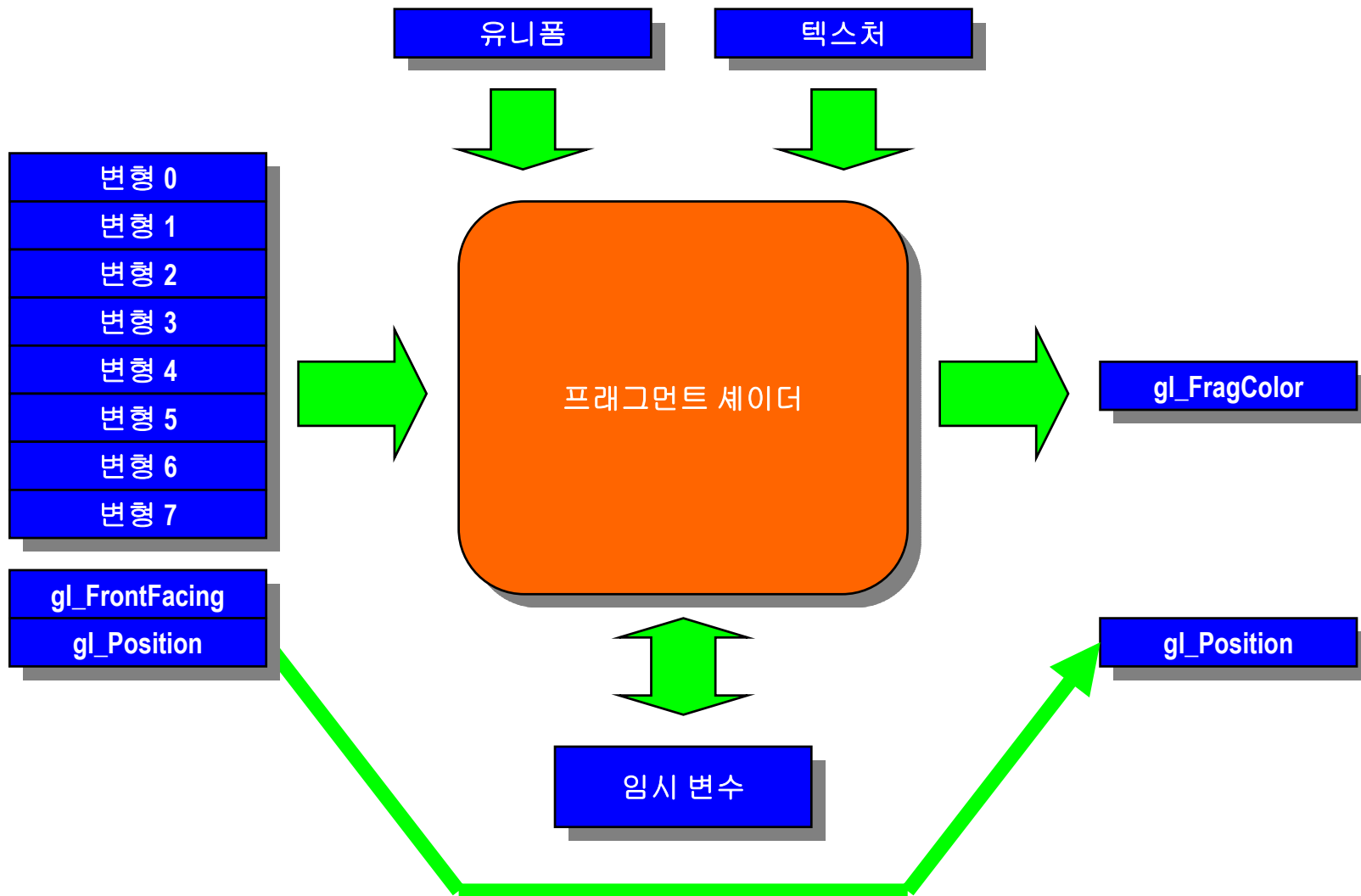
프로그래머의 모델



버텍스 셰이더



프래그먼트 셰이더



GLSL(OpenGL Shading Language)

- ‘C’와 유사한 언어
- 형식:
 - void float vec2 vec3 vec4 mat2 mat3 mat4 int ivec2 ivec3 ivec4 bool bvec2 bvec3 bvec4 samplerND
 - 구조체, 배열
- 형식 한정자
 - uniform
 - ‘상수’(조명 위치/방향, 텍스처 단위, ...)
 - 버텍스에 따라 변경되지 않음
 - attribute
 - 버텍스 단위 값(위치, 법선, ...)
 - varying
 - 버텍스 셰이더가 생성
 - 래스터라이즈에 의한 보간으로 픽셀 단위 값 생성
 - 프래그먼트 셰이더의 입력으로 사용
 - 예: 텍스처 좌표

GLSL 계속

- 연산자

- ++ -- + - ! () []
- * / + -
- < <= > >=
- == !=
- && ^^ ||
- ?:
- = *= /= += -=

- 스위즐(Swizzle)

- 예: vector.xyzw vector.xxxx vector.wxyz ...

- 흐름 제어

- <expression> ? <expression_1> : <expression_2>
- if, else, for, while, do, return, break, continue, discard(프래그먼트 셰이더 전용)

GLSL 기본 제공 변수

- **ES**의 목표는 고정된 기능을 줄이는 것입니다.
 - 가장 이상적인 모델은 완전한 프로그램 가능 모델
 - 여전히 고정된 기능 존재(계속 정의 중)
- 버텍스 셰이더
 - `vec4 gl_Position;` // Write only (required)
- 프래그먼트 셰이더
 - `float gl_FragDepth;` // Read-only
 - `vec4 gl_FragCoord;` // Read-only
 - `bool gl_FrontFacing;` // Read-only
 - `float gl_FragColor;` // Write only (required)

GLSL 기본 제공 함수

- 일반 함수
 - pow, exp2, log2, sqrt, inversesqrt
 - abs, sign, floor, ceil, fract, mod, min, max, clamp
- 삼각 함수
 - radians, degrees, sin, cos, tan, asin, acos, atan
- 기하
 - length, distance, cross, dot, normalize, faceForward, reflect
- 행렬
 - matrixCompMult
- 보간
 - **mix**(x,y,alpha)
 $x * (1.0 - \alpha) + y * \alpha$
 - **step**(edge,x)
 $x \leq \text{edge} ? 0.0 : 1.0$
 - **smoothstep**(edge0,edge1,x)
 $t = (x - \text{edge0}) / (\text{edge1} - \text{edge0});$
 $t = \text{clamp}(t, 0.0, 1.0);$
 $\text{return } t * t * (3.0 - 2.0 * t);$
- 벡터 비교
 - lessThan, lessThanEqual, greaterThan, greaterThanEqual, equal, notEqual, notEqual, any, all
- 텍스처
 - texture1D, texture2D, texture3D, textureCube
 - texture1DProj, texture2DProj, textureCubeProj

버텍스 셰이더 함수

- 버텍스 셰이더로 처리할 수 있는 작업:
 - 모델-뷰 및 투영 행렬을 사용하여 위치 변환
 - 법선 변환(재규격화 포함)
 - 텍스처 좌표 생성 및 변환
 - 버텍스 단위 조명
 - 픽셀 단위 조명 값 계산
- 버텍스 셰이더로 처리할 수 없는 작업:
 - 두 개 이상의 버텍스 정보가 필요한 작업
 - 연결에 종속된 작업
 - 삼각 함수 연산(예: 클리핑, 선별)
 - 색상 버퍼 액세스

버텍스 셰이더 예제

- 난반사광

```
uniform mat4 ModelViewProjectionMatrix, NormalMatrix;  
uniform vec4 LightSourceDiffuse, LightSourcePosition, MaterialDiffuse;
```

```
attribute vec4 InputPosition, InputNormal, InputTextureCoordinates;
```

```
varying vec4 VertexColour;  
varying vec4 TextureCoordinates;
```

```
void main()  
{  
    vec3 normal, lightDirection;  
    vec4 diffuse;  
    float NdotL;  
  
    normal = normalize(NormalMatrix * Normal);  
    lightDirection = normalize(vec3(LightSourcePosition));  
    NdotL = max(dot(normal, lightDirection), 0.0);  
    diffuse = MaterialDiffuse * LightSourceDiffuse;  
    VertexColor = NdotL * diffuse;  
  
    TextureCoordinates = InputTextureCoordinates;  
  
    gl_Position = ModelViewProjectionMatrix * position;  
}
```

프래그먼트 셰이더 함수

- 프래그먼트 셰이더로 처리할 수 있는 작업:

- 텍스처 혼합
- 안개
- 알파 테스트
- 종속 텍스처
- 픽셀 무시
- 범프 및 환경 매핑

- 프래그먼트 셰이더로 처리할 수 없는 작업:

- 색상 버퍼를 사용한 혼합
- ROP 연산
- 깊이 또는 스텐실 테스트
- 쓰기 깊이

프래그먼트 셰이더 예제

- 단순 텍스처 혼합

```
uniform sampler2D TextureHandle;  
varying vec2 TextureCoordinates;  
varying vec4 VertexColour;  
void main()  
{  
    vec4 texel = texture2D (TextureHandle, TextureCoordinates);  
    gl_FragColor = texel * VertexColour;  
}
```

Gooch 셰이더(버텍스)

```
uniform vec4 lightPos;
```

```
varying vec3 normal;
```

```
varying vec3 lightVec;
```

```
varying vec3 viewVec;
```

```
void main()
```

```
{
```

```
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
```

```
    vec4 vert = gl_ModelViewMatrix * gl_Vertex;
```

```
    normal = gl_NormalMatrix * gl_Normal;
```

```
    lightVec = vec3(lightPos - vert);
```

```
    viewVec = -vec3(vert);
```

```
}
```



Gooch 셰이더(프래그먼트)

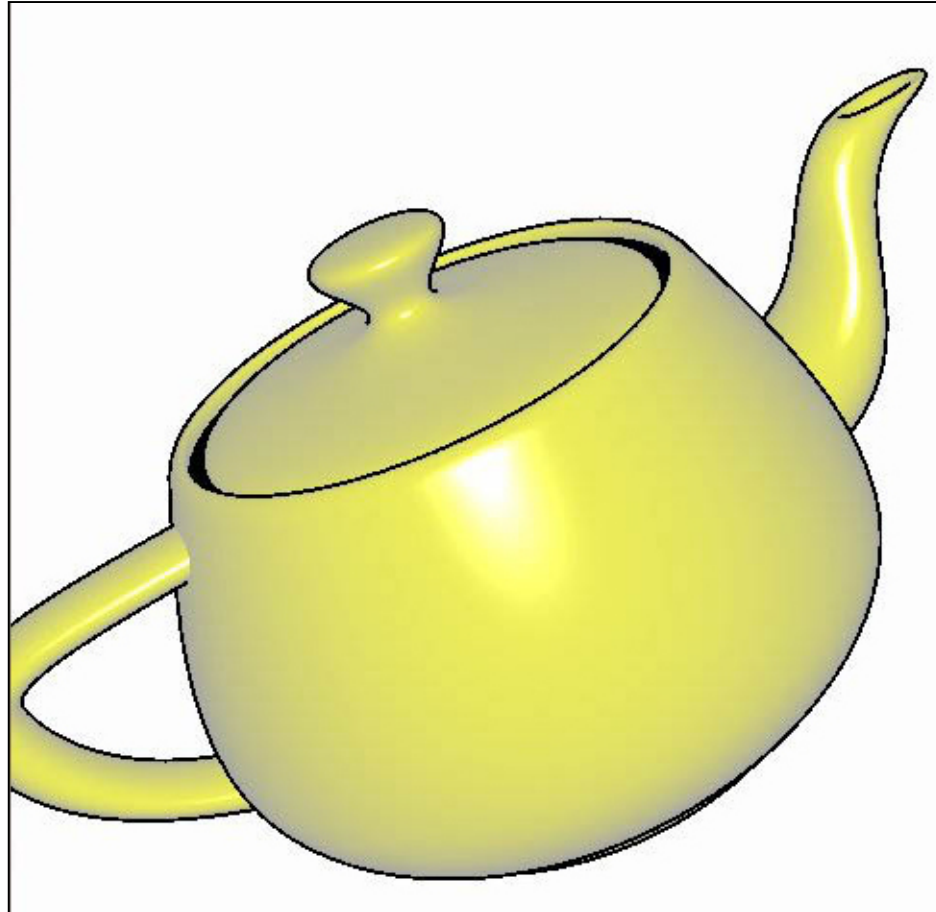
```
uniform vec3 ambient;
```

```
varying vec3 normal;  
varying vec3 lightVec;  
varying vec3 viewVec;
```

```
void main(){  
    const float b = 0.55;  
    const float y = 0.3;  
    const float Ka = 1.0;  
    const float Kd = 0.8;  
    const float Ks = 0.9;  
  
    vec3 specularcolor = vec3(1.0, 1.0, 1.0);  
  
    vec3 norm = normalize(normal);  
    vec3 L = normalize (lightVec);  
    vec3 V = normalize (viewVec);  
    vec3 halfAngle = normalize (L + V);  
    vec3 orange = vec3(.88,.81,.49);  
    vec3 purple = vec3(.58,.10,.76);  
  
    vec3 kCool = purple;  
    vec3 kWarm = orange;  
  
    float NdotL = dot(L, norm);  
    float NdotH = clamp(dot(halfAngle, norm), 0.0, 1.0);  
    float specular = pow(NdotH, 64.0);  
  
    float blendval = 0.5 * NdotL + 0.5;  
    vec3 Cgooch = mix(kWarm, kCool, blendval);  
  
    vec3 result = Ka * ambient + Kd * Cgooch + specularcolor * Ks * specular;  
  
    gl_FragColor = vec4(result, 1.0);  
}
```

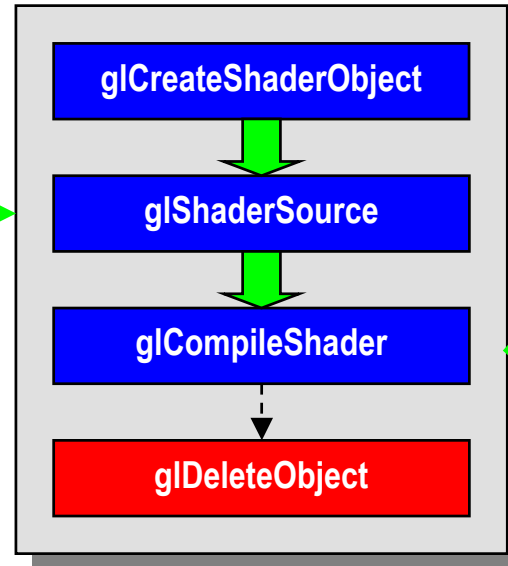
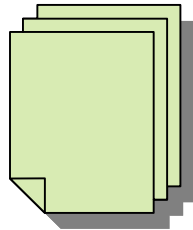


Gooch 셰이더

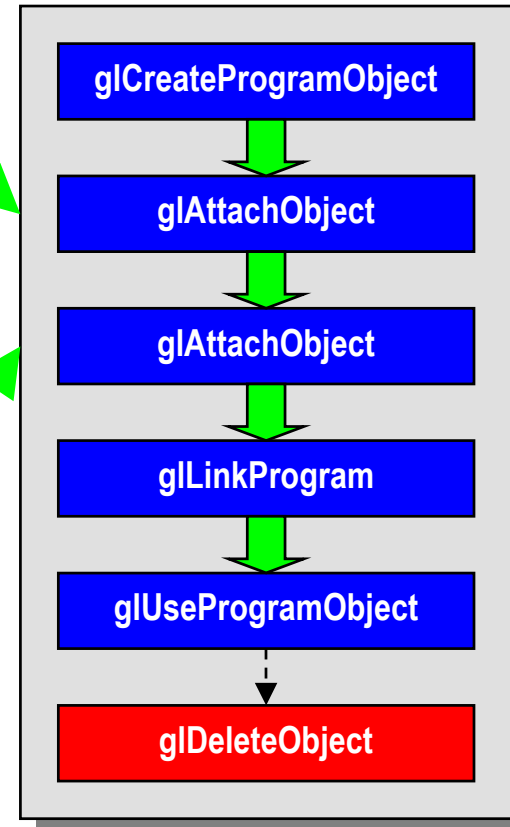
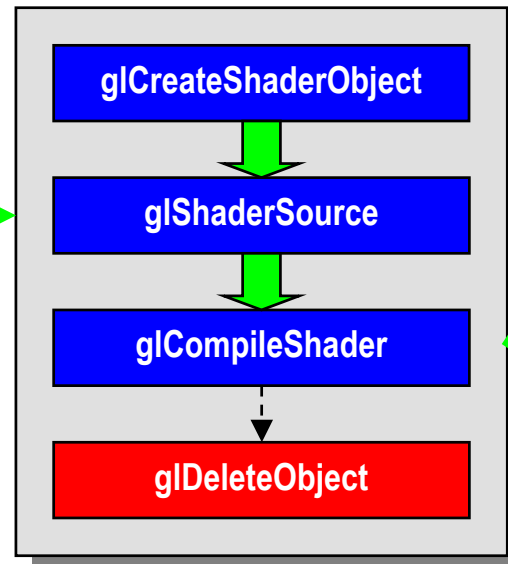
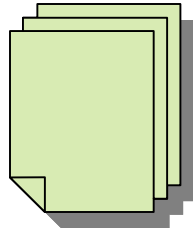


셰이더 컴파일 및 사용

버텍스 셰이더



프래그먼트 셰이더



셰이더 컴파일 및 사용: 코드

// Create Shader Objects

```
GLhandleARB programObject = glCreateProgramObjectARB();
```

```
GLhandleARB vertexShaderObject = glCreateShaderObjectARB(GL_VERTEX_SHADER_ARB);
```

```
GLhandleARB fragmentShaderObject = glCreateShaderObjectARB(GL_FRAGMENT_SHADER_ARB);
```

```
GLcharARB *vertexShaderSource = readShaderFile(vertexShaderFilename);
```

```
GLcharARB *fragmentShaderSource = readShaderFile(fragmentShaderFilename);
```

// Load code into shader objects

```
glShaderSourceARB(vertexShaderObject, 1, vertexShaderSource, NULL);
```

```
glShaderSourceARB(fragmentShaderObject, 1, fragmentShaderSource, NULL);
```

```
glCompileShaderARB(vertexShaderObject);
```

```
glCompileShaderARB(fragmentShaderObject);
```

```
glAttachObjectARB(programObject, vertexShaderObject);
```

```
glAttachObjectARB(programObject, fragmentShaderObject);
```

```
glLinkProgramARB(programObject);
```

```
glUseProgramObjectARB(programObject);
```

성능 팁

- 프래그먼트 셰이더 간소화

- 프래그먼트 셰이더 하드웨어는 비용이 많이 듭니다.
- 초기 제품은 복잡한 셰이더에서 좋은 성능을 보여 주지 못했습니다.

- 텍스처를 사용한 함수 조회 감소

- 계산 비용은 저렴하지만 텍스처 액세스 비용이 많이 듭니다.
- 임베디드 장치에서 더욱 중요합니다.

- 레지스터 사용량 최소화

- 임베디드 장치는 데스크톱 장치보다 적은 수의 레지스터를 지원합니다. 레지스터를 메모리로 확장하는 것은 비용이 많이 듭니다.

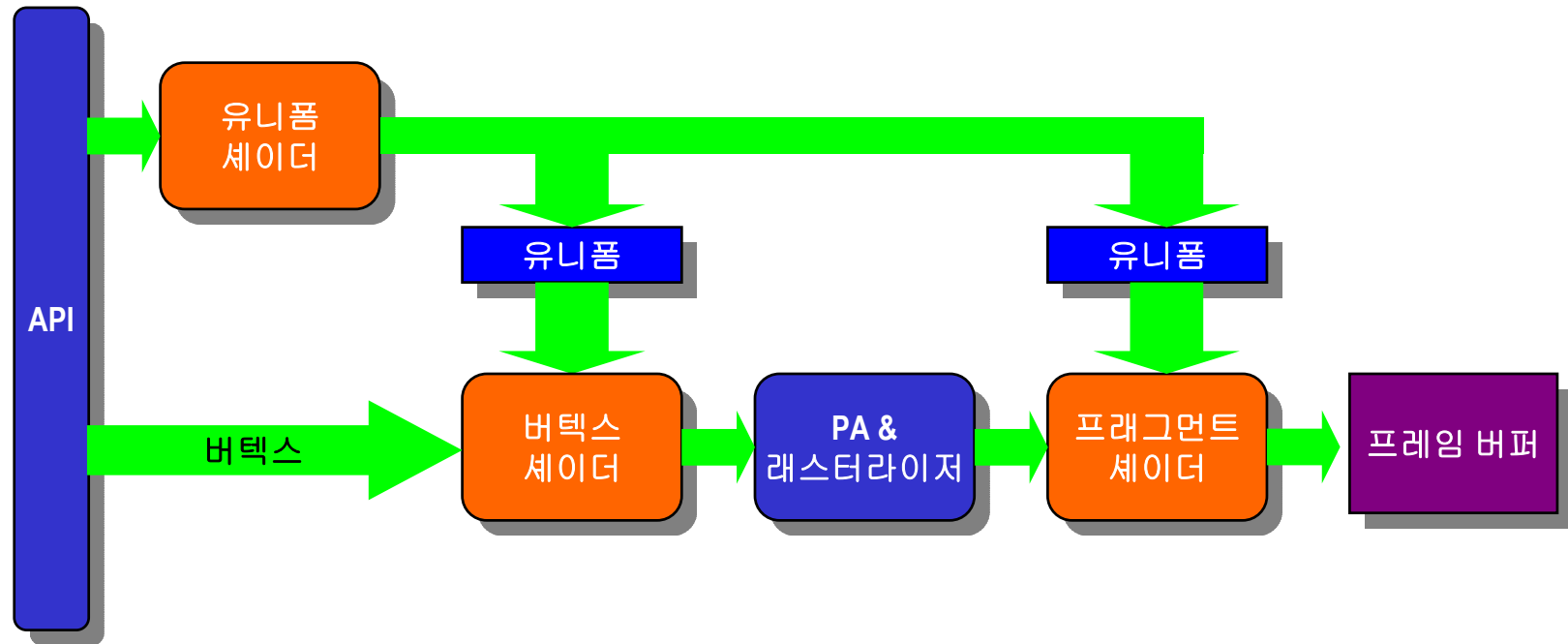
- 셰이더 변경 횟수 최소화

- 셰이더에 많은 상태가 포함되어 있습니다.
- 파이프라인을 플러시해야 할 수도 있습니다.
- 새 셰이더를 로드하는 기본 설정에서 유니폼을 사용하여 동작을 변경합니다.

유니폼 셰이더

- 질문: 유니폼을 계산하는 방법은?
 - 예: 역행렬을 생성하는 방법
- 문제:
 - API에서 기존의 고정 함수를 제거하려고 합니다.
 - 임베디드 장치의 CPU가 느리고 부동 소수점이 지원되지 않습니다.
- 가능한 해결책:
 - 버텍스 셰이더에 코드를 배치하고 컴파일러를 사용하여 코드를 ‘가져와’ 한 번만 실행합니다.
 - 고정 함수를 다시 배치합니다.
 - 유니폼 셰이더
- **ES2.0**에 대해서는 아직 결정되지 않음

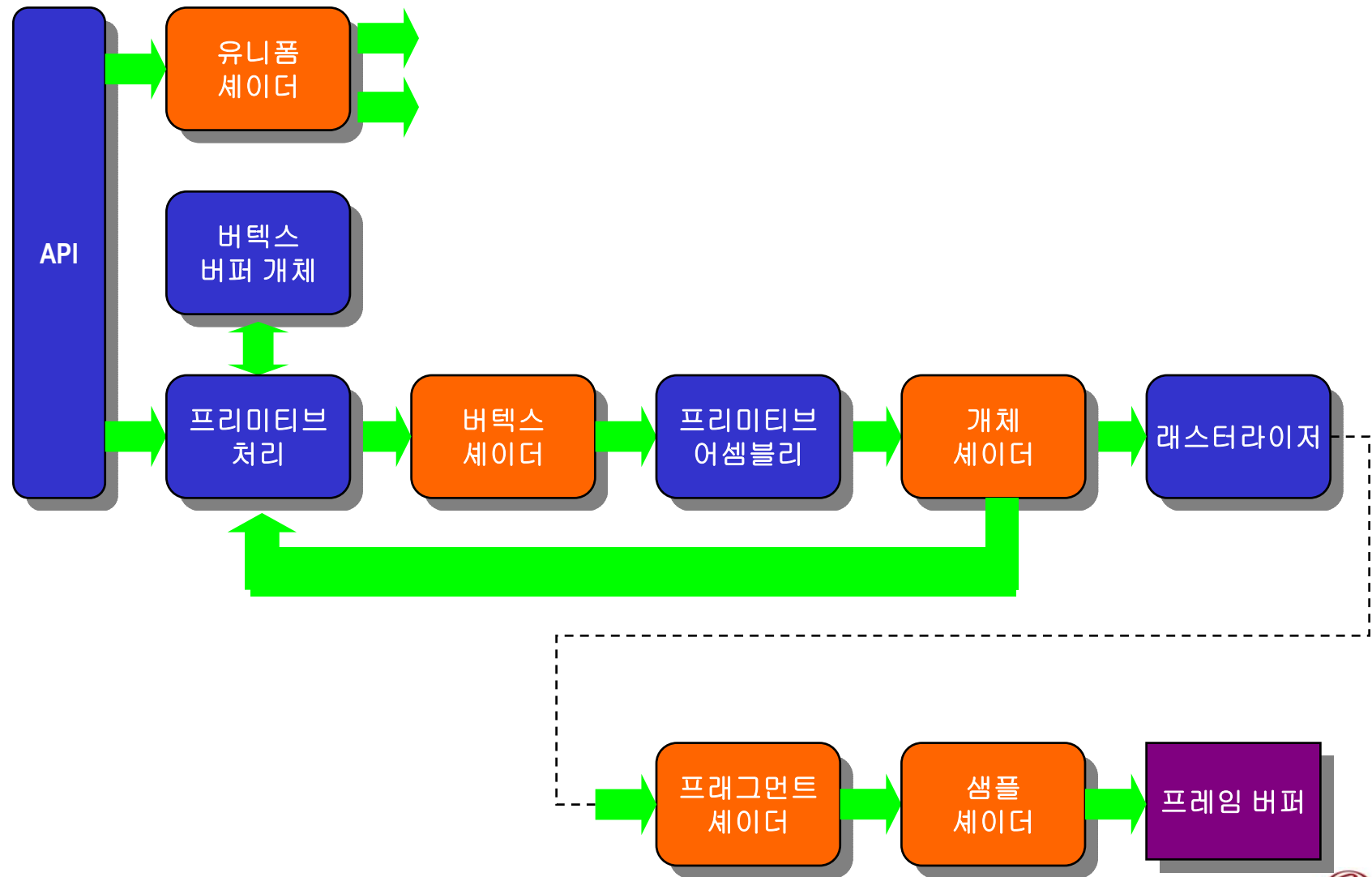
유니폼 셰이더



앞으로의 방향

- 단순한 셰이더
 - 더 많은 고정 함수 파이프라인 제거
 - 다중 샘플링 사용 시 더 많은 프로그램 가능 기능 제공
 - 예: 읽기/쓰기 깊이 및 스텐실
- 개체 셰이더
 - 프로그램 가능 테셀레이션(tessellation)
 - 교차 표면
 - Procedural Geometry 방식
 - 음영 처리, 교차 선별 같은 많은 알고리즘 속도 향상

미래의 **ES** 파이프라인



질문이 있으십니까?



© Copyright Khronos Group, 2005 - 28페이지

